

MileStone #2

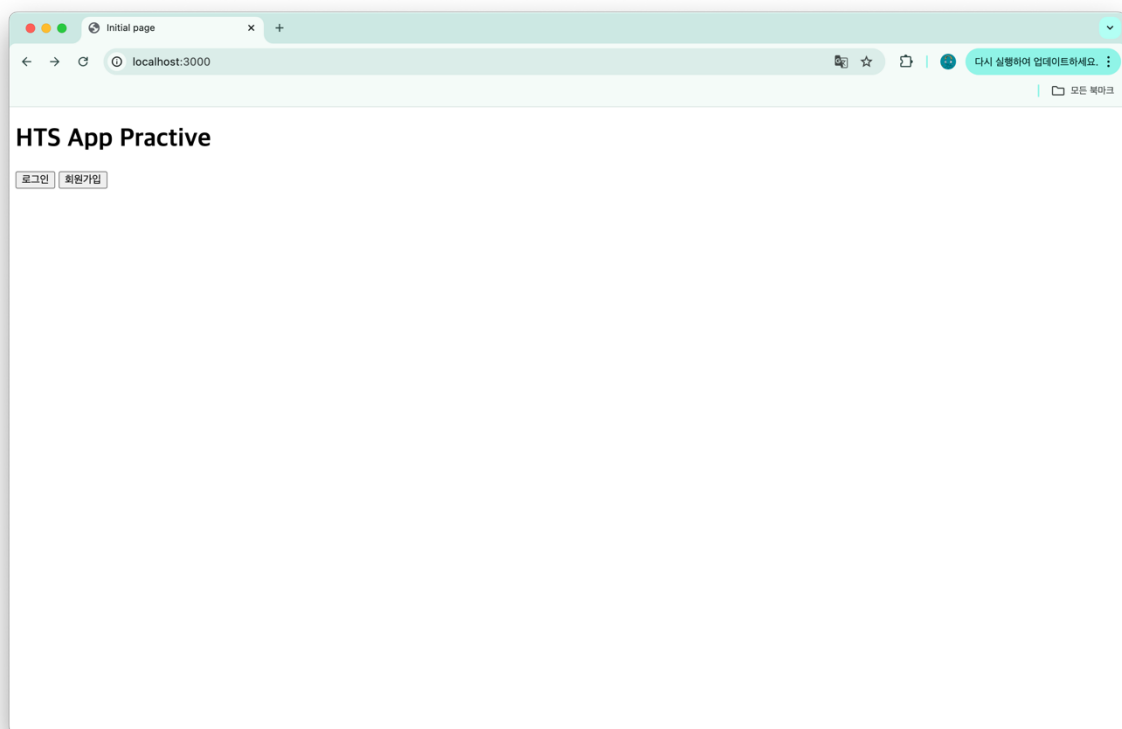
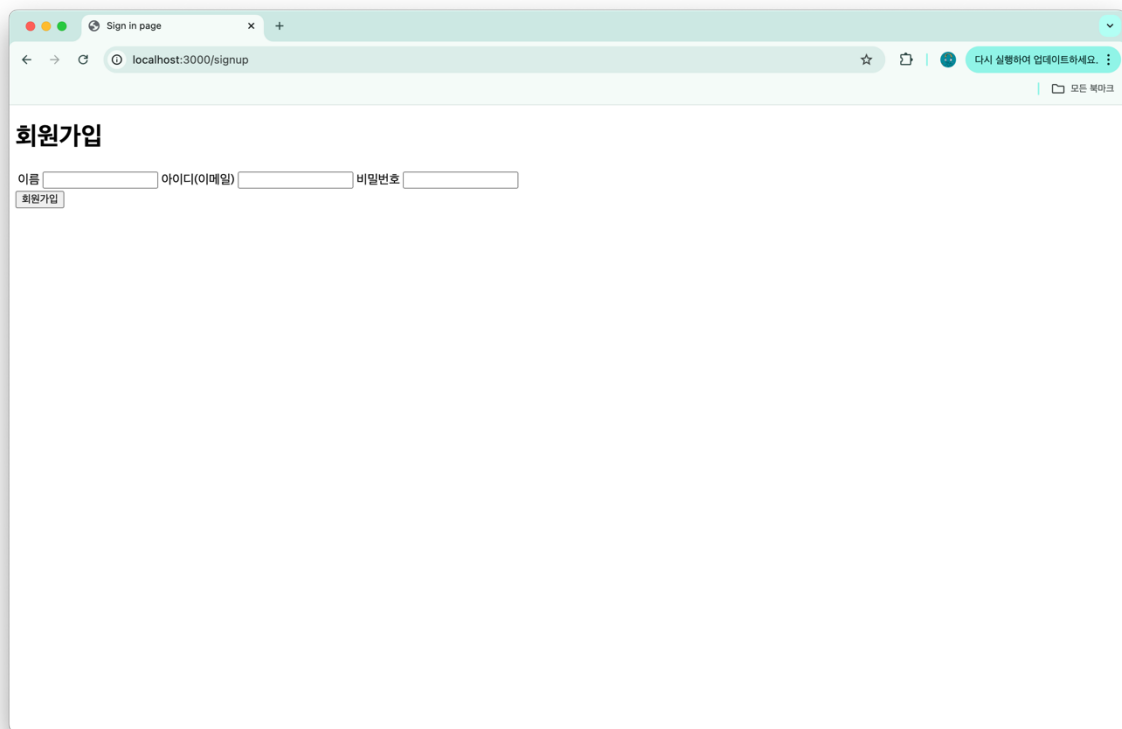
HTS

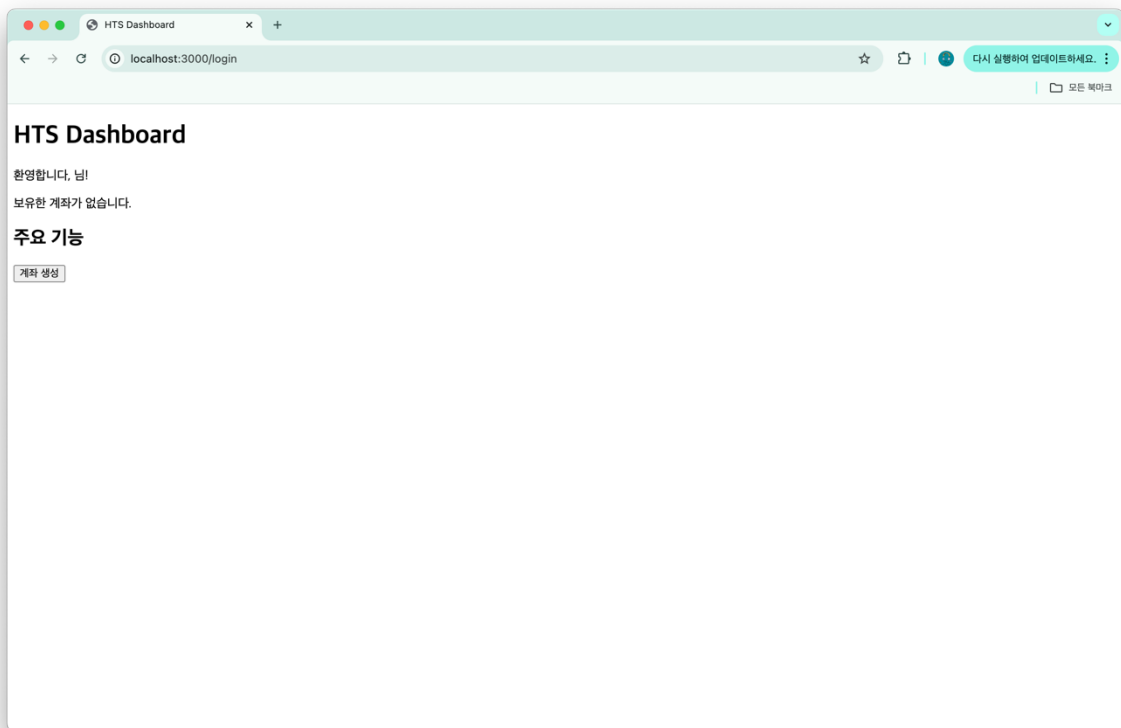
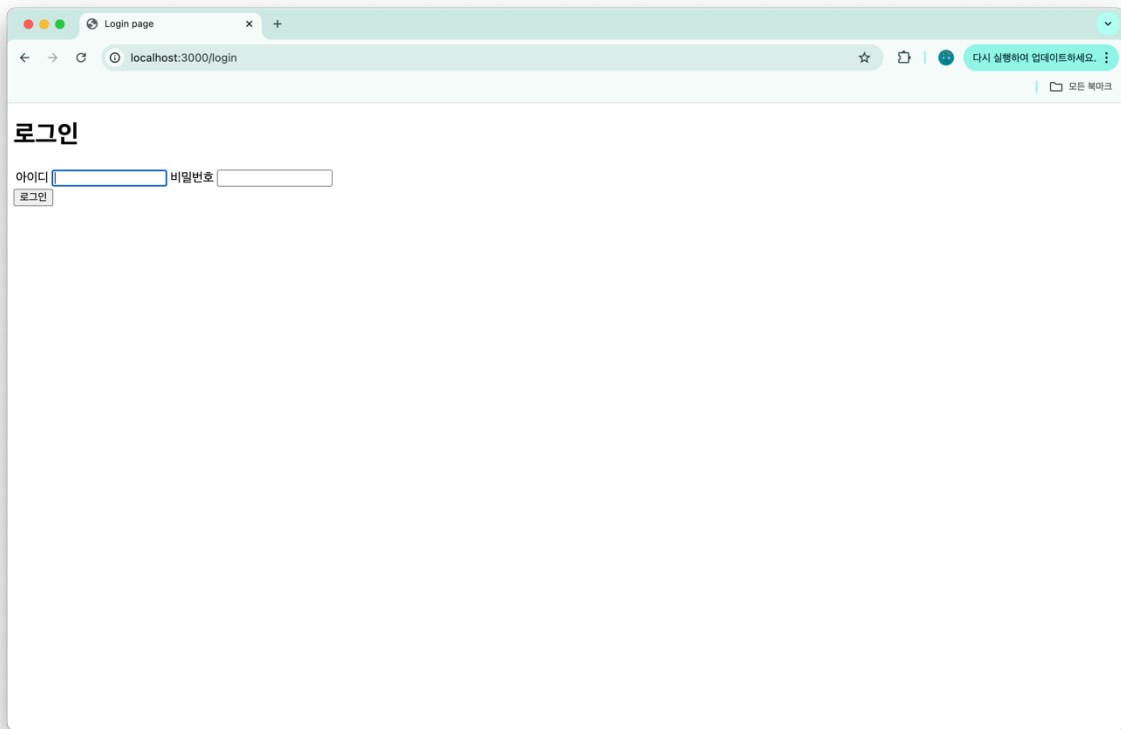
2019060546

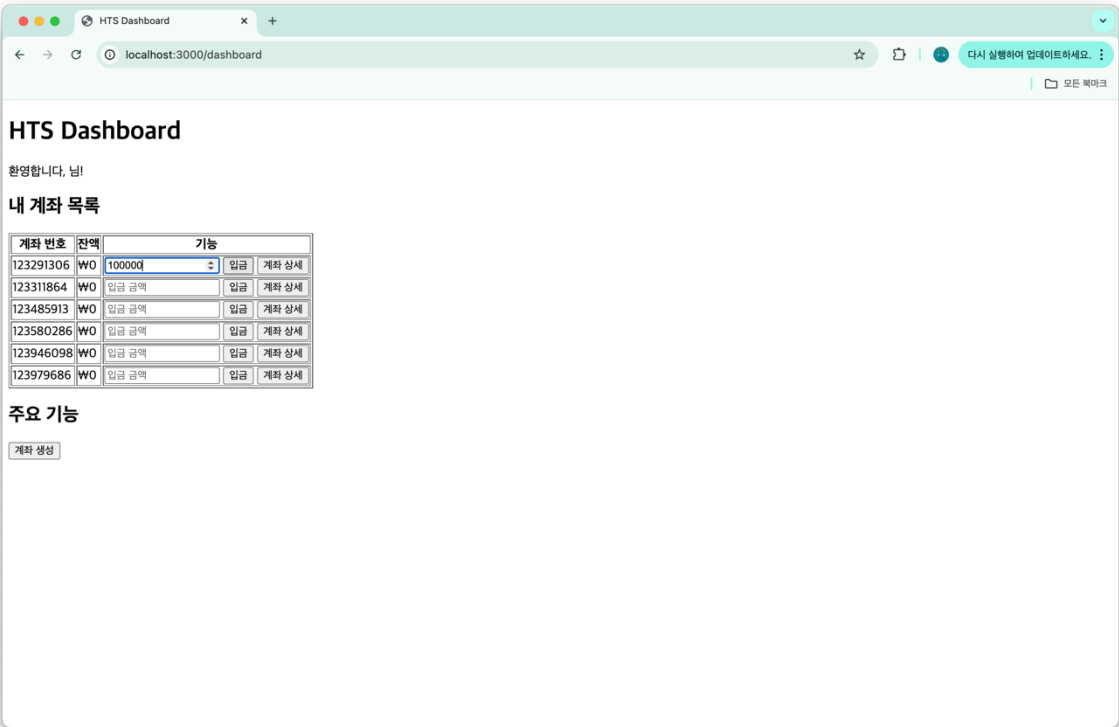
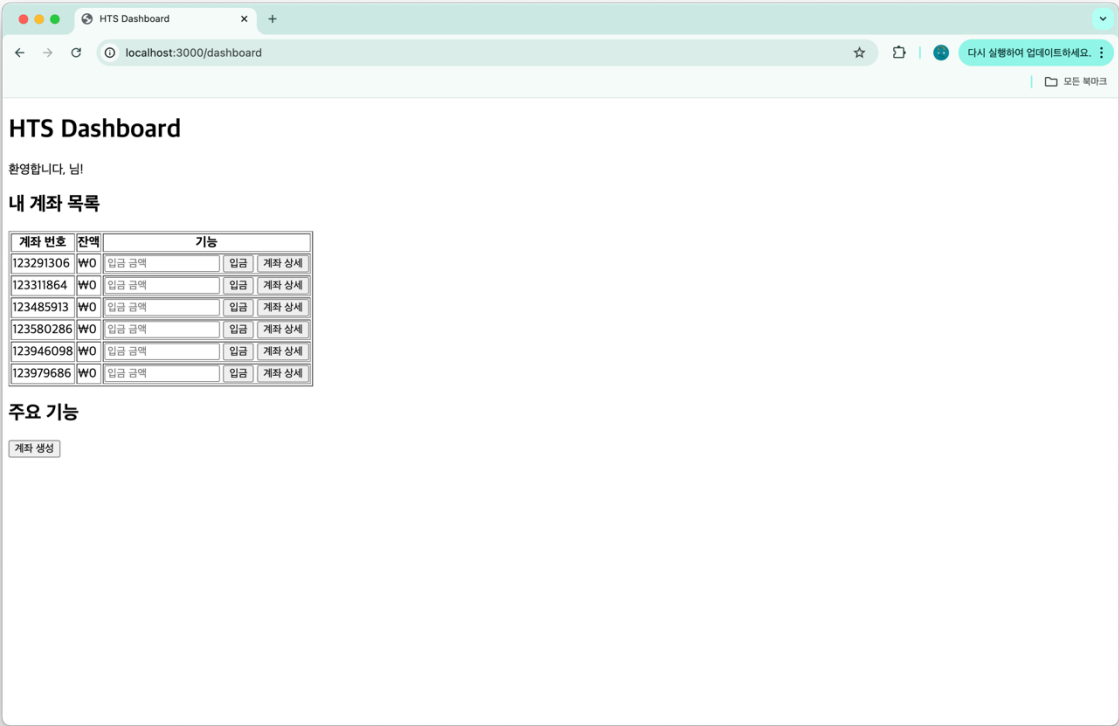
주원웅

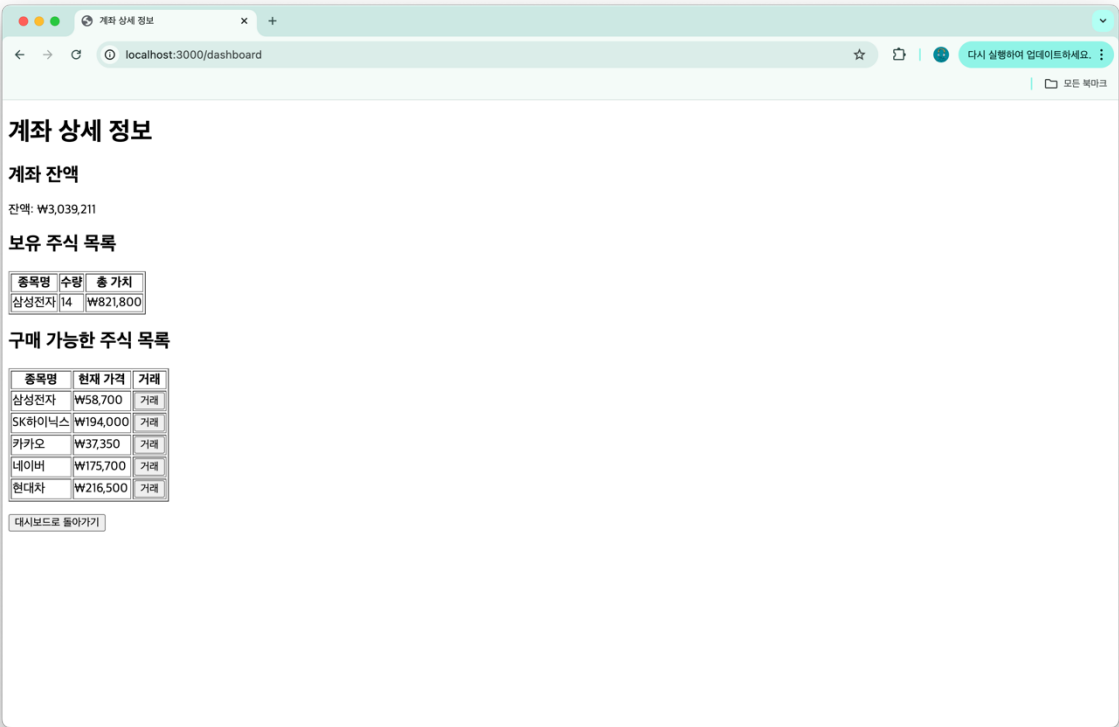
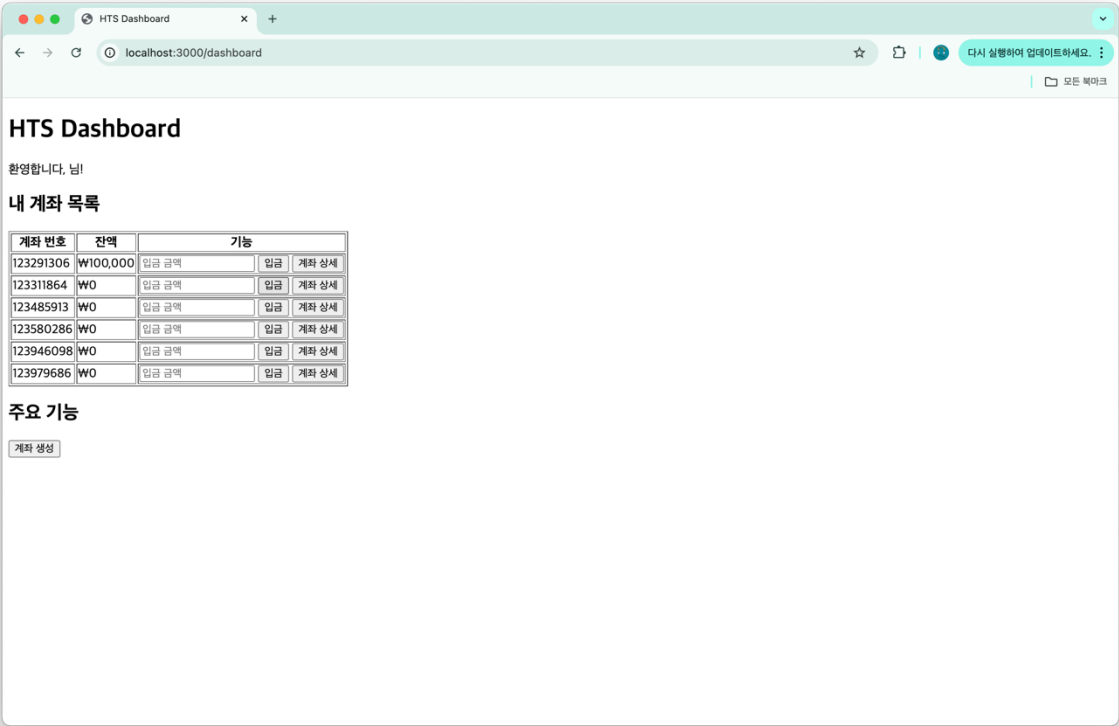
Youtube

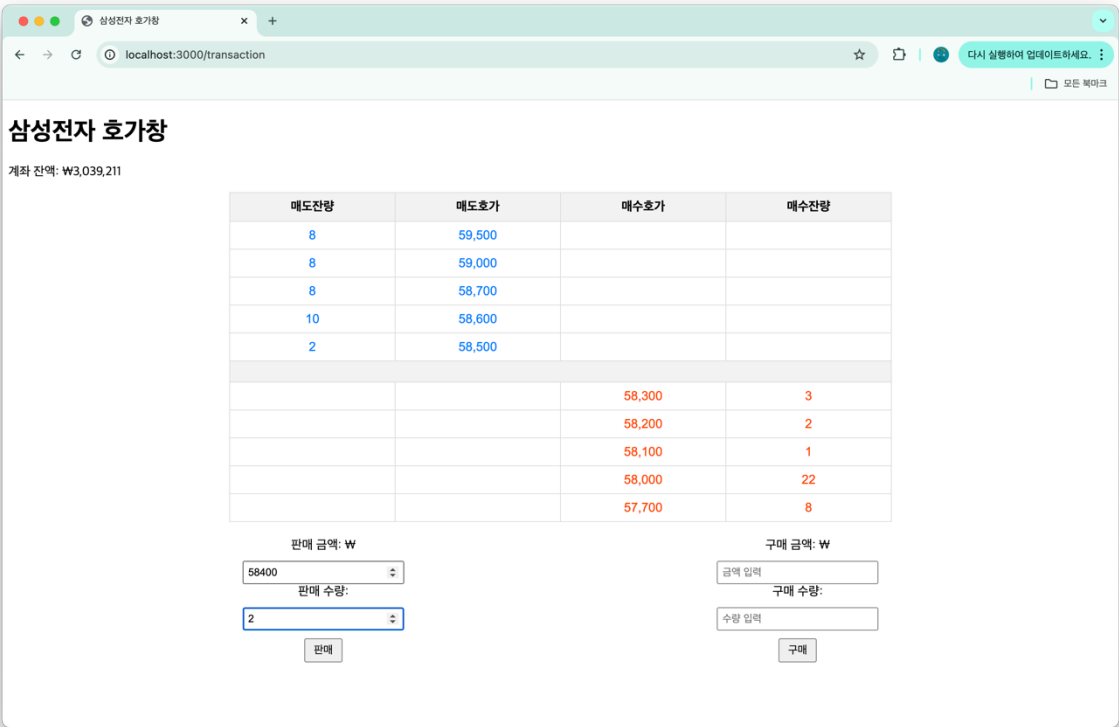
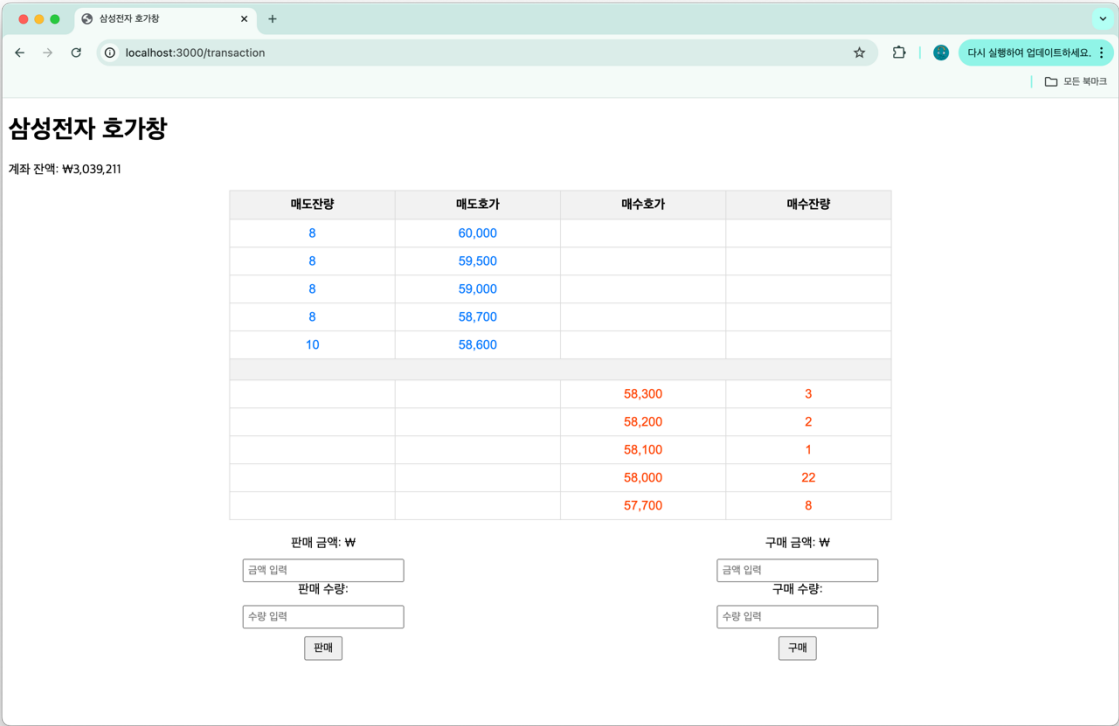
<https://youtu.be/63A8T3oolTU>











삼성전자 호가창

계좌 잔액: ₩3,039,211

매도잔량	매도호가	매수호가	매수잔량
8	59,000		
8	58,700		
10	58,600		
2	58,500		
2	58,400		
		58,300	3
		58,200	2
		58,100	1
		58,000	22
		57,700	8

판매 금액: ₩

금액 입력

판매 수량:

수량 입력

판매

구매 금액: ₩

금액 입력

구매 수량:

수량 입력

구매

삼성전자 호가창

계좌 잔액: ₩3,039,211

매도잔량	매도호가	매수호가	매수잔량
8	59,000		
8	58,700		
10	58,600		
2	58,500		
2	58,400		
		58,300	3
		58,200	2
		58,100	1
		58,000	22
		57,700	8

판매 금액: ₩

58300

판매 수량:

3

판매

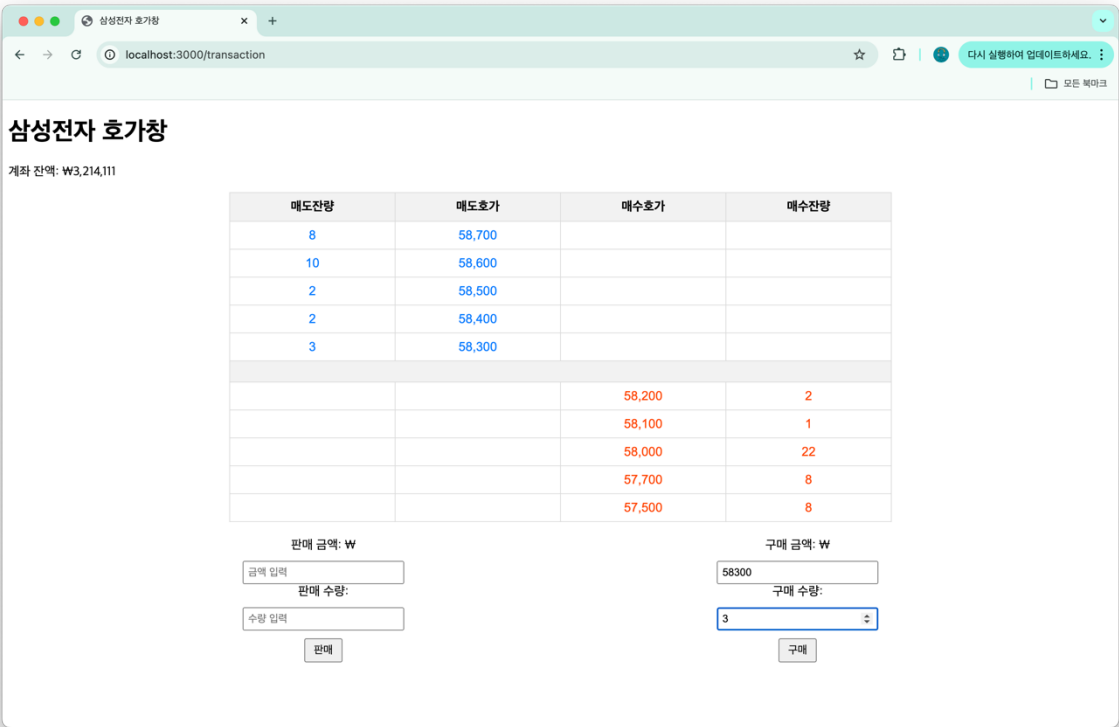
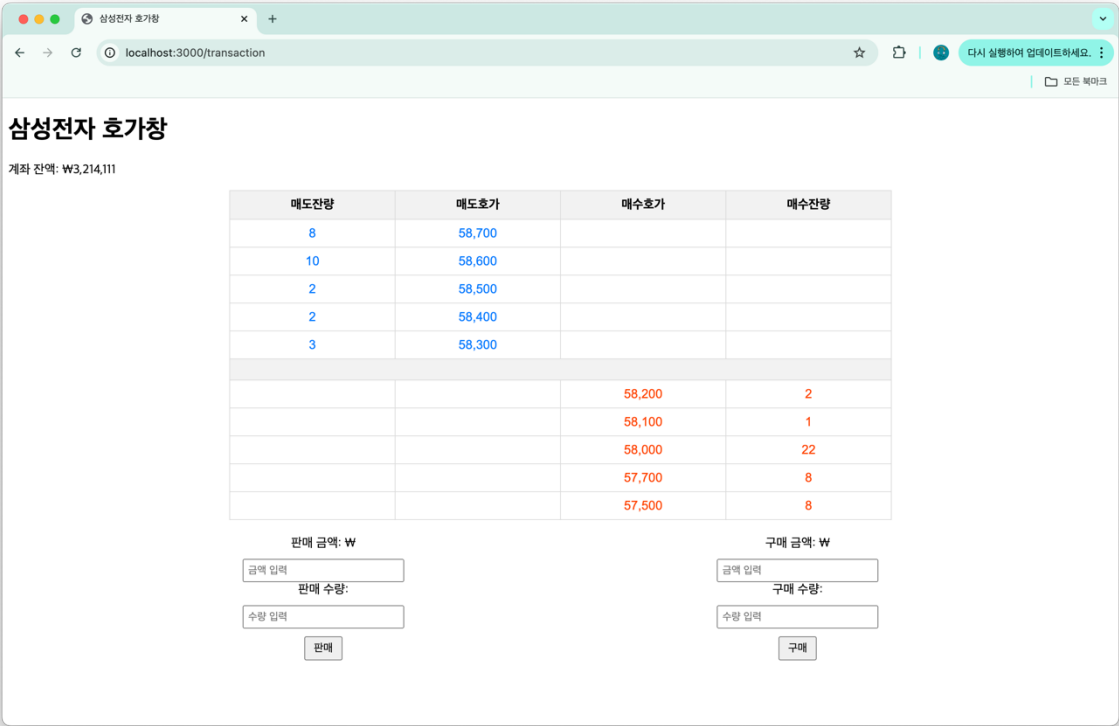
구매 금액: ₩

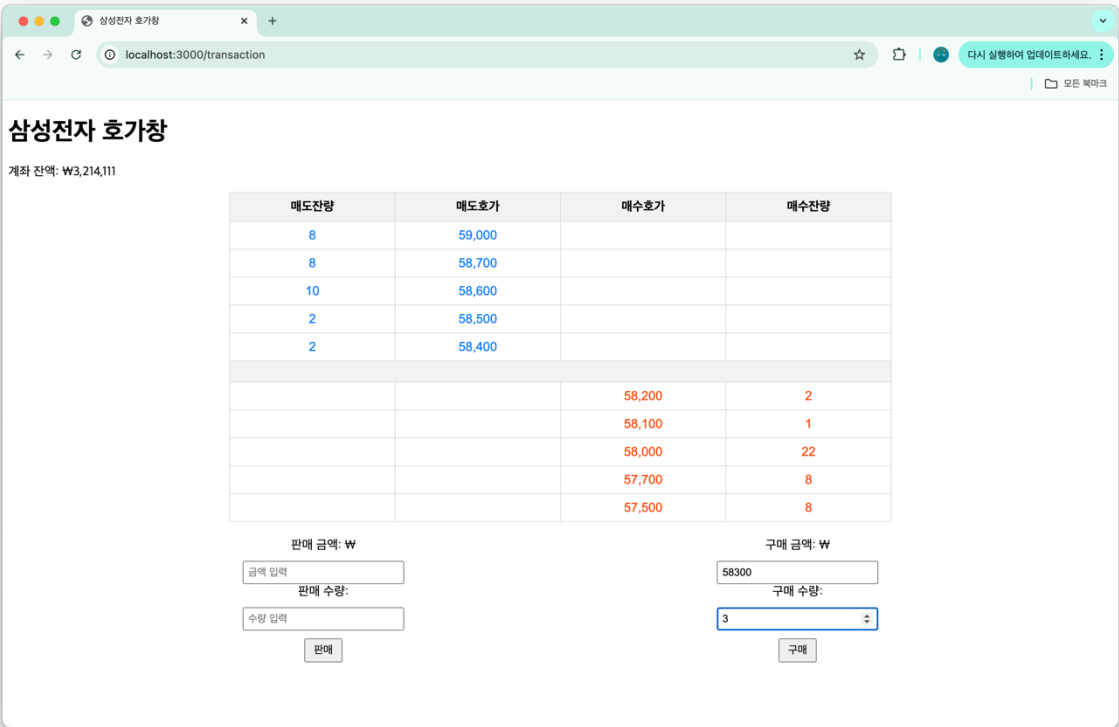
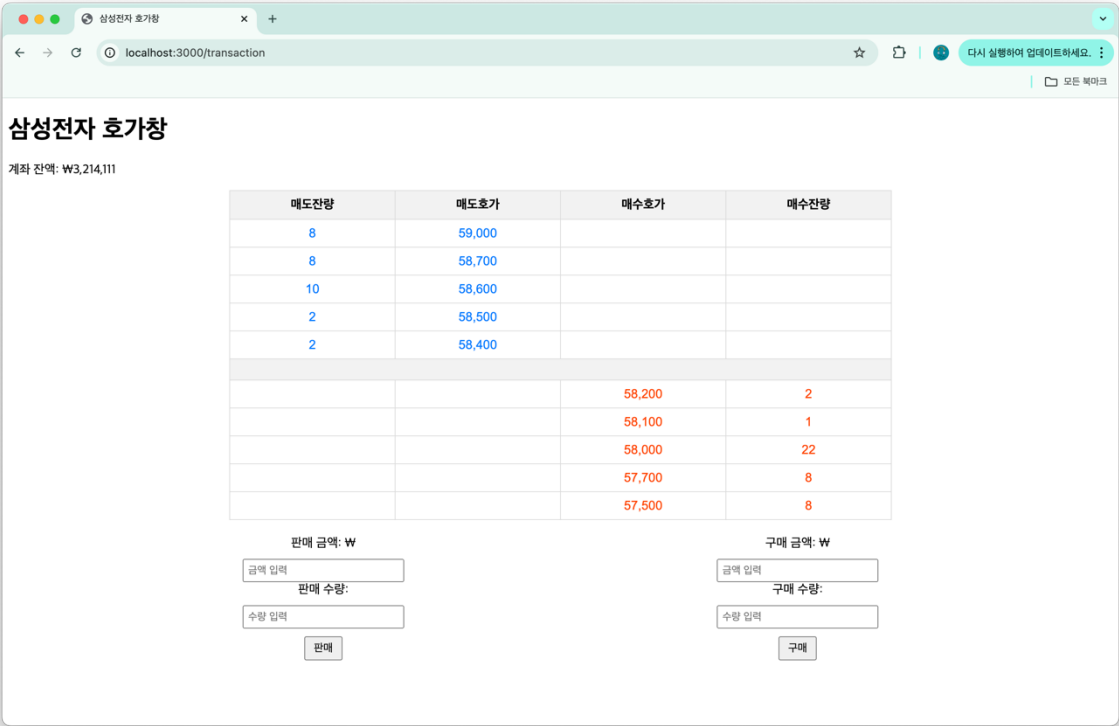
금액 입력

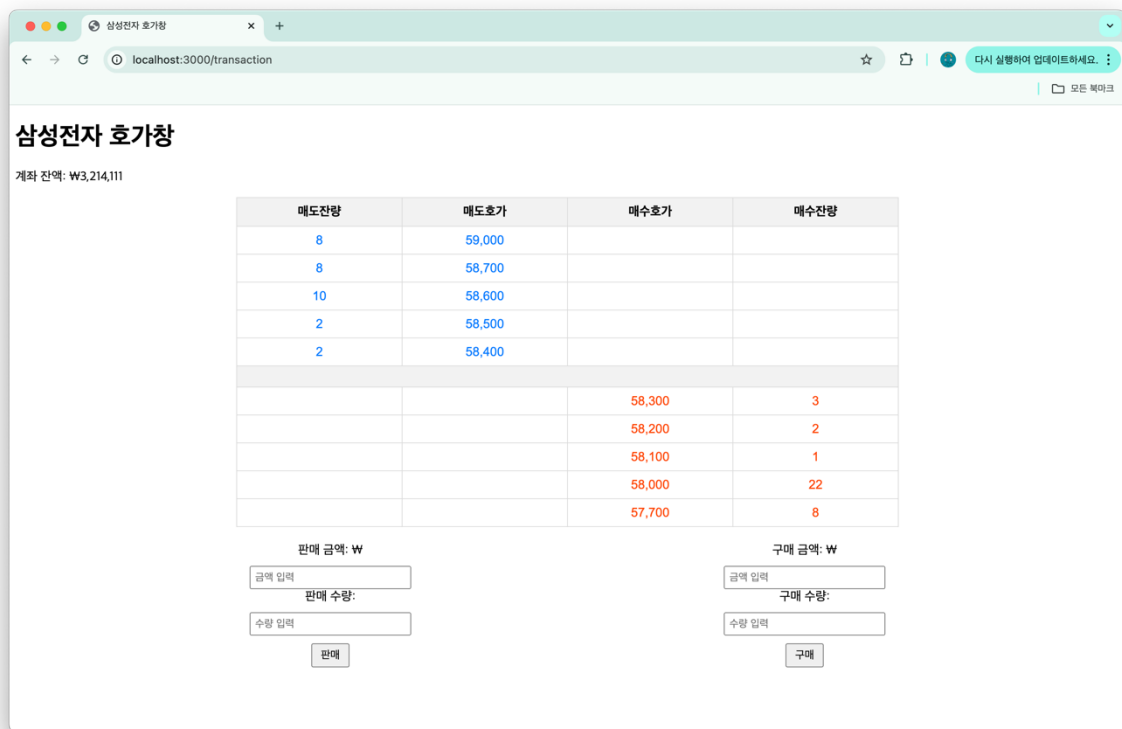
구매 수량:

수량 입력

구매







Design

회원가입, 로그인, 계좌목록(대시보드), 계좌 상세(주식 거래), 호가창 페이지를 만들 예정이다. 이렇게 구성한 이유는 사용자마다 각자 고유 계정이 있어야 하고, 사용자가 가지고 있는 계좌를 통해 주식거래가 성사되도록 하기 위함이다.

Implement

1. Sample data

```
- const initStock = "SELECT * FROM stocks";
- sql_connection.query(initStock, (error, results, fields) => {
-   if (error) throw error;
-   if (results == 0) {
-     let i = 1;
-     sql_connection.query("insert into stocks (stock_id, stock_name,
-       current_price) values(?, '삼성전자', 58700)", [i++], (error, results1, fields)
-       => { if (error) throw error; });
```

```

-
-         sql_connection.query("insert into stocks (stock_id, stock_name,
current_price) values(?, 'SK 하이닉스', 194000)", [i++], (error, results1,
fields) => { if (error) throw error; });
-
-         sql_connection.query("insert into stocks (stock_id, stock_name,
current_price) values(?, '카카오', 37350)", [i++], (error, results1, fields)
=> { if (error) throw error; });
-
-         sql_connection.query("insert into stocks (stock_id, stock_name,
current_price) values(?, '네이버', 175700)", [i++], (error, results1, fields)
=> { if (error) throw error; });
-
-         sql_connection.query("insert into stocks (stock_id, stock_name,
current_price) values(?, '현대차', 216500)", [i++], (error, results1, fields)
=> { if (error) throw error; });
-     }
- });

```

- Stock 의 샘플을 넣어주는 sql

- insert into transactions (account_id, stock_id, transaction_date, transaction_status, transaction_type, trade_type, quantity, price) values(123447201,1,NOW(), 'Pending', 'Buy', 'Limit', 10, 58000);

- sample 로 transaction(거래) 생성하는 sql 이 sql의 경우 transaction 의 account_id attribute 가 foreign key 이기 때문에, 계좌를 생성한 이후에 해당 계좌에 알맞은 계좌번호가 존재해야 샘플 생성이 가능하다.

- 여러 transaction Sample 을 통해서 호가창을 구성한다.

2. 서버 sql

app.post("/login")

1.쿼리: "SELECT id FROM users WHERE email = ? AND password = ?"

- 사용자가 입력한 계정과 비밀번호가 users 테이블의 데이터와 일치하는지 확인하여 로그인 인증을 수행합니다.

2.쿼리: "SELECT account_id, cash_balance FROM stock_accounts WHERE user_id = ?"

- 사용자가 로그인 성공 시, 해당 사용자의 계좌 목록과 잔액 정보를 stock_accounts 테이블에서 조회합니다.

app.post("/dashboard")

- **createAccount 액션**

1. 쿼리: "INSERT INTO stock_accounts (account_id, user_id, cash_balance) VALUES (?, ?, ?)"

- 목적: 사용자가 계좌를 생성할 때 stock_accounts 테이블에 새로운 계좌 정보를 삽입합니다.

2. 쿼리: "SELECT account_id, cash_balance FROM stock_accounts WHERE user_id = ?"

- 목적: 계좌 생성 후 최신 계좌 목록을 조회하여 대시보드에 반영합니다.

- **deposit 액션**

1. 쿼리: "UPDATE stock_accounts SET cash_balance = cash_balance + ? WHERE account_id = ?"

- 목적: 사용자가 입금할 금액을 기존 계좌 잔액에 추가하여 stock_accounts 테이블을 업데이트합니다.

2. 쿼리: "SELECT account_id, cash_balance FROM stock_accounts WHERE user_id = ?"

- 목적: 입금 후 최신 계좌 목록을 조회하여 대시보드에 반영합니다.

- **detail 액션**

1. 쿼리: "SELECT cash_balance FROM stock_accounts WHERE account_id = ?"

- 목적: 특정 계좌의 현금 잔액을 조회합니다.

2. 쿼리: holdings 테이블과 stocks 테이블을 조인하여 보유 주식 목록을 조회하는 쿼리

- 목적: 보유 중인 주식의 현재 수량과 총 가치를 조회합니다.

3. 쿼리: "SELECT * FROM stocks"

- 목적: stocks 테이블에서 모든 주식 정보를 가져옵니다.

app.post("/account")

1. 쿼리: "SELECT account_id, cash_balance FROM stock_accounts WHERE user_id = ?"

- 목적: 사용자의 모든 계좌 목록을 조회하여 대시보드로 이동할 때 사용합니다.

app.post("/transaction")

- **transaction 액션 (transaction 페이지 불러오기)**

1. 쿼리: "SELECT cash_balance FROM stock_accounts WHERE account_id = ?"

- 목적: 선택한 계좌의 현금 잔액을 조회합니다.
- 이유: 거래 페이지에 들어가기 전에 계좌의 현금 잔액을 확인하여 사용자에게 보여줍니다.

2. 쿼리: buyquery = `SELECT stock_id, price, SUM(quantity) AS total_quantity,
transaction_type, transaction_status`

FROM transactions

GROUP BY stock_id, price, transaction_type, transaction_status

HAVING transaction_type = 'Buy' AND transaction_status = 'Pending' AND stock_id = ?

ORDER BY price DESC

LIMIT 5`

- 목적: transactions 테이블에서 Pending 상태의 Buy 주문을 조회하여 매수 거래 목록을 가져옵니다.

3. 쿼리: sellquery = `SELECT stock_id, price, SUM(quantity) AS total_quantity,
transaction_type, transaction_status`

FROM transactions

GROUP BY stock_id, price, transaction_type, transaction_status

HAVING transaction_type = 'Buy' AND transaction_status = 'Pending' AND stock_id = ?

ORDER BY price DESC

LIMIT 5`

- 목적: transactions 테이블에서 Pending 상태의 Sell 주문을 조회하여 매도 거래 목록을 가져옵니다.

- **buy** 액션

1. 쿼리: buy_req_query

- 목적: Pending 상태의 Sell 주문 중 가장 낮은 가격의 매물을 조회합니다.

2. 쿼리: "INSERT INTO transactions ..."

- 목적: 사용자가 원하는 매수 주문을 transactions 테이블에 삽입합니다.

3. 쿼리: "UPDATE stock_accounts SET cash_balance = cash_balance - ? WHERE account_id = ?"

- 목적: 매수 후 계좌의 현금 잔액을 차감합니다.

4. 쿼리: "UPDATE transactions SET quantity = ? WHERE account_id = ? AND stock_id = ? AND transaction_date = ?"

- 목적: 매수량만큼 매도자의 주문 수량을 업데이트합니다.

5. 쿼리: "UPDATE stock_accounts SET cash_balance = cash_balance + ? WHERE account_id = ?"

- 목적: 매도자 계좌의 현금 잔액에 매수 금액을 추가합니다.

6. 쿼리: holdings 테이블 업데이트

- 목적: 사용자가 매수한 주식을 holdings 테이블에 추가하거나 기존 수량에 더합니다.

- **sell** 액션

1. 쿼리: "SELECT quantity FROM holdings WHERE account_id = ? AND stock_id = ?"

- 목적: 사용자가 매도하려는 주식의 보유 수량을 확인합니다.

2. 쿼리: sell_req_query

- 목적: Pending 상태의 Buy 주문 중 가장 높은 가격의 매물을 조회합니다.

3. 쿼리: "INSERT INTO transactions ..."

- 목적: 사용자가 원하는 매도 주문을 transactions 테이블에 삽입합니다.

4. 쿼리: "UPDATE holdings SET available_quantity = available_quantity - ?, quantity = quantity - ? WHERE account_id = ? AND stock_id = ?"

- 목적: 매도 후 보유 주식의 수량을 차감합니다.

5. 쿼리: "UPDATE transactions SET quantity = ? WHERE account_id = ? AND stock_id = ? AND transaction_date = ?"

- 목적: 매도량만큼 매수자의 주문 수량을 업데이트합니다.

6. 쿼리: holdings 테이블 업데이트

- 목적: 사용자가 매도한 주식을 holdings 테이블에 추가하거나 기존 수량에 더합니다.

Trouble Shooting

- 거래 체결 방식이 매우 복잡했다.
- Transaction이 여러 row 로 겹쳐 있는 거래 체결의 경우 수행이 복잡했는데, 해결하지 못했다.